



# LDD3 Ch. 3

# Char Drivers

ECEN 427

**BYU** Electrical & Computer  
Engineering  
IRA A. FULTON COLLEGE OF ENGINEERING

# Learning Outcomes

- Trace how `open()` on a path in `/dev` reaches a driver: device file, major/minor, `cdev`, `file_operations`
- Allocate device numbers, register a `cdev` with a `file_operations` table, and create the `/dev` node from the driver
- Implement `open`, `release`, `read` and `write`, and carry per-device state between them with `private_data`
- Move data between user and kernel space with `copy_to_user` and `copy_from_user`, and explain why a user pointer can't be dereferenced directly

The textbook walks through the creation of a driver called ***scull***.

- “scull is a char driver that acts on a memory area as though it were a device.”

The scull source implements the following devices:

*scull0 to scull3*

Four devices, each consisting of a memory area that is both global and persistent. Global means that if the device is opened multiple times, the data contained within the device is shared by all the file descriptors that opened it. Persistent means that if the device is closed and reopened, data isn't lost. This device can be fun to work with, because it can be accessed and tested using conventional commands, such as *cp*, *cat*, and shell I/O redirection.

*scullpipe0 to scullpipe3*

Four FIFO (first-in-first-out) devices, which act like pipes. One process reads what another process writes. If multiple processes read the same device, they contend for data. The internals of *scullpipe* will show how blocking and non-blocking *read* and *write* can be implemented without having to resort to interrupts. Although real drivers synchronize with their devices using hardware interrupts, the topic of blocking and nonblocking operations is an important one and is separate from interrupt handling (covered in Chapter 10).

*scullsingle*

*scullpriv*

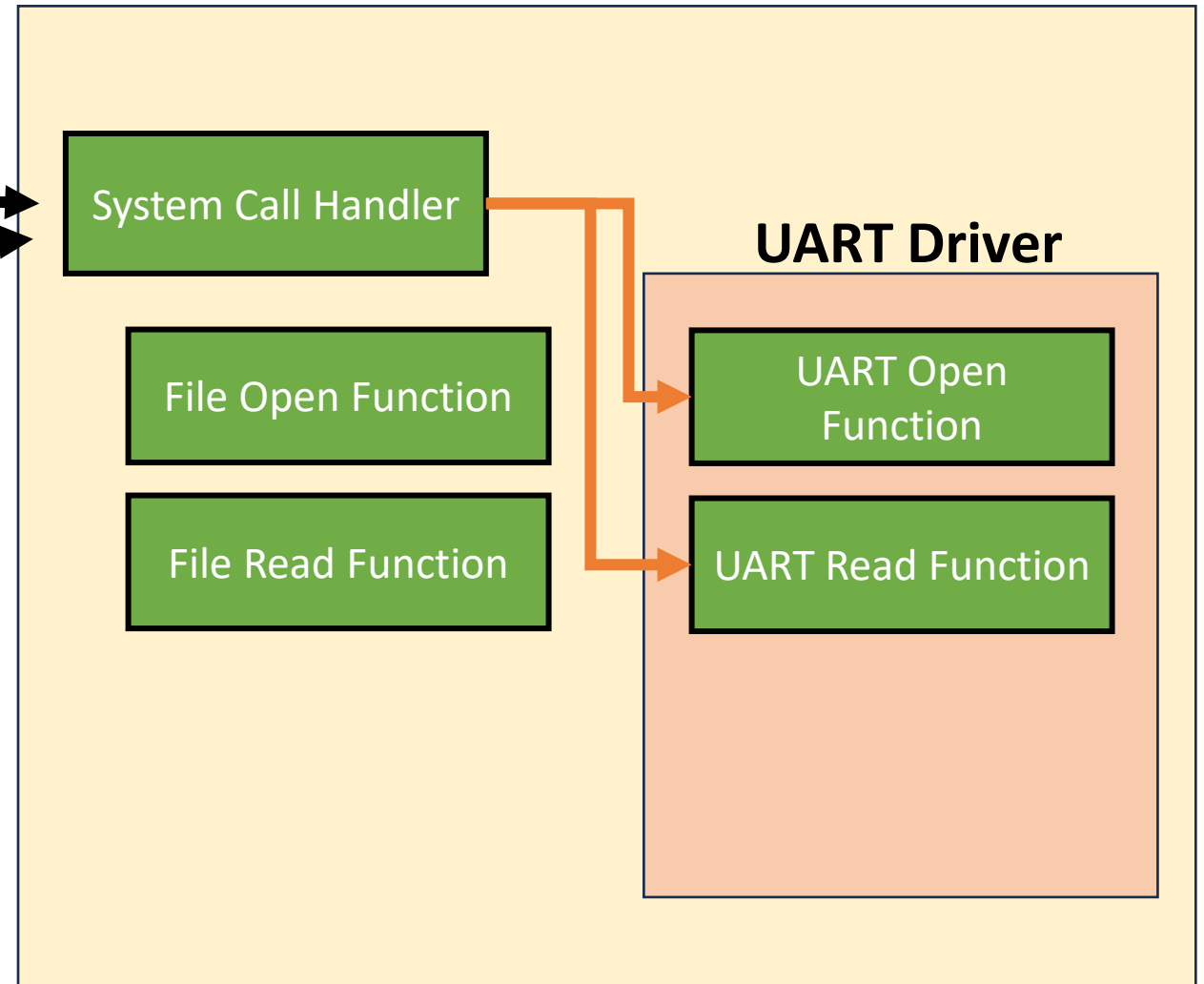
# REVIEW: Linux System Calls

## Userspace

```
int main() {  
    ...  
    int fd = open("/dev/ttyUSB1")  
    read(fd, ...)   
    ...  
}
```

**How does `open("/dev/...")`  
know to connect to the  
appropriate driver module?**

## Kernel Code



# What's in /dev?

```
byu@pynq:~$ ls -la /dev/
total 4
drwxr-xr-x 14 root root    3300 Sep 16 10:52 .
drwxr-xr-x 19 root root    4096 Sep  3 10:08 ..
drwxr-xr-x  2 root root     580 Sep 16 09:53 block
crw-rw----  1 root disk 10, 234 Sep 16 09:53 btrfs-control
drwxr-xr-x  3 root root      60 Dec 31 1969 bus
drwxr-xr-x  2 root root   2520 Sep 16 10:52 char
crw-----  1 root root    5,  1 Sep 16 09:55 console
crw-----  1 root root 10, 125 Sep 16 09:53 cpu_dma_latency
drwxr-xr-x  7 root root    140 Sep 16 09:53 disk
crw-rw-rw-  1 root root 242,  0 Sep 16 09:53 dma_desc
drwxr-xr-x  3 root root    100 Sep 16 09:53 dri
drwxr-xr-x  2 root root    200 Sep 16 10:52 ecen427
crw-rw-rw-  1 root root 243,  0 Sep 16 09:53 ecen427_hdmi
lrwxrwxrwx  1 root root     13 Apr  7 2022 fd -> /proc/self/fd
crw-----  1 root root 10, 126 Sep 16 09:53 fpga0
crw-rw-rw-  1 root root  1,  7 Sep 16 09:53 full
crw-----  1 root root 254,  0 Sep 16 09:53 gpiochip0
crw-rw----  1 root i2c   89,  0 Sep 16 09:53 i2c-0
crw-rw----  1 root i2c   89,  1 Sep 16 09:53 i2c-1
crw-----  1 root root 247,  0 Sep 16 09:53 iio:device0
lrwxrwxrwx  1 root root     12 Apr  7 2022 initctl -> /run/initctl
```

A device file has no contents

It is a name attached to a pair of numbers:

a **major** and **minor** number

The **major** number identifies the module/**driver** associated with the device.

The **minor** number is used to index the **device**

*(remember, drivers can manage multiple devices)*

Within the kernel, the `dev_t` type (defined in `<linux/types.h>`) is used to hold device numbers—both the major and minor parts. As of Version 2.6.0 of the kernel, `dev_t` is a 32-bit quantity with 12 bits set aside for the major number and 20 for the minor number. Your code should, of course, never make any assumptions about the internal organization of device numbers; it should, instead, make use of a set of macros found in `<linux/kdev_t.h>`. To obtain the major or minor parts of a `dev_t`, use:

```
MAJOR(dev_t dev);  
MINOR(dev_t dev);
```



**Get** major/minor  
from `dev_t`

If, instead, you have the major and minor numbers and need to turn them into a `dev_t`, use:

```
MKDEV(int major, int minor);
```



**Create** `dev_t` from  
major/minor  
numbers.

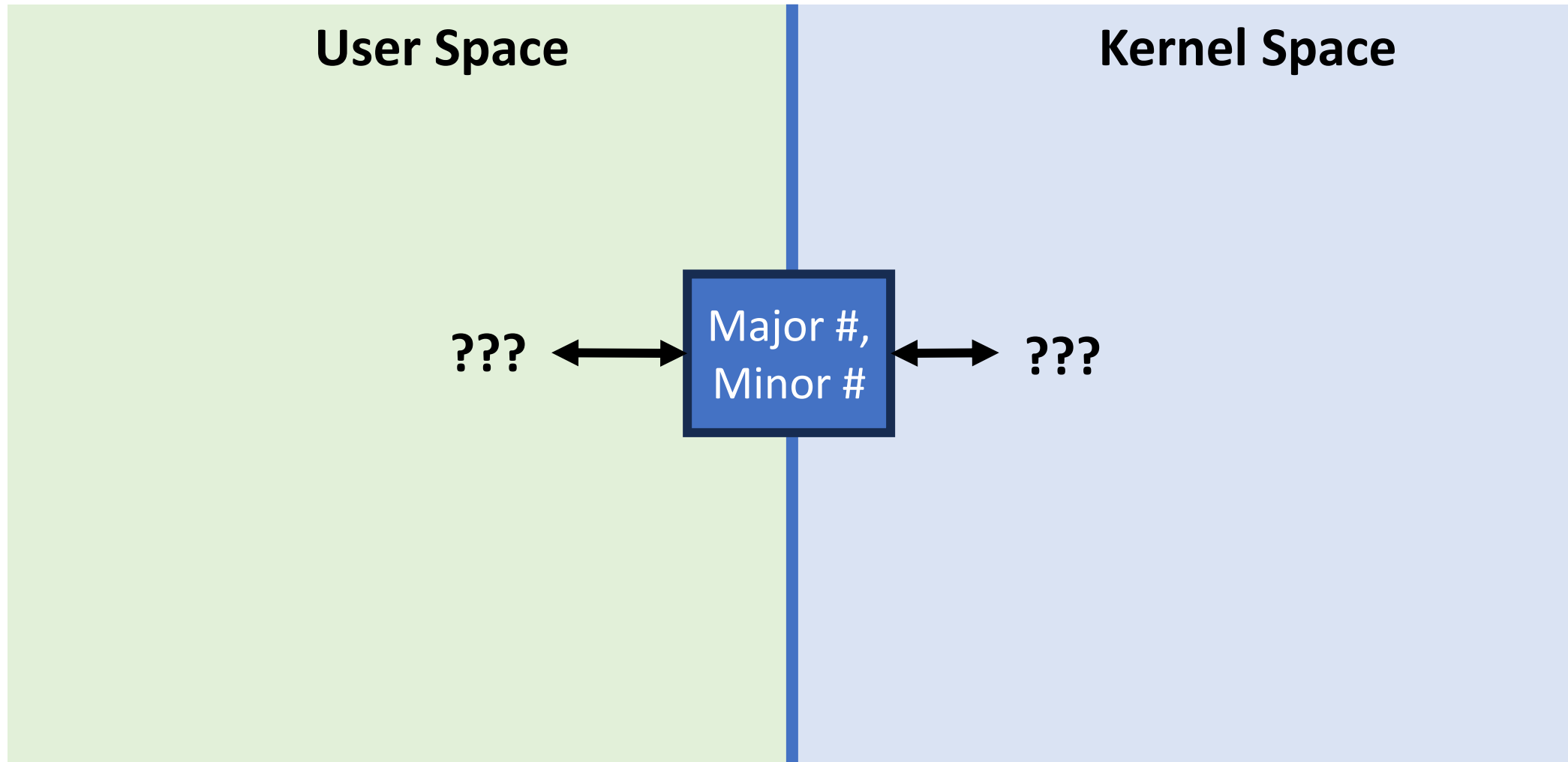
# Allocating Device Numbers

```
int alloc_chrdev_region(dev_t *dev, unsigned int firstminor,  
                        unsigned int count, char *name);
```

- With this function, **dev** is an output-only parameter that will, on successful completion, hold the first number in your allocated range.
- **firstminor** should be the requested first minor number to use; it is usually 0.
- **count** is the total number of contiguous device numbers you are requesting.
- Finally, **name** is the name of the device that should be associated with this number range

Regardless of how you allocate your device numbers, you should free them when they are no longer in use. Device numbers are freed with:

```
void unregister_chrdev_region(dev_t first, unsigned int count);
```



The following script, *scull\_load*, is part of the *scull* distribution. The user of a driver that is distributed in the form of a module can invoke such a script from the system's *rc.local* file or call it manually whenever the module is needed.

```
#!/bin/sh
module="scull"
device="scull"
mode="664"

# invoke insmod with all arguments we got
# and use a pathname, as newer modutils don't look in . by default
/sbin/insmod ./module.ko $* || exit 1

# remove stale nodes
rm -f /dev/${device}[0-3]

major=$(awk "\\$2==\\\"$module\\\" {print \\$1}" /proc/devices)

mknod /dev/${device}0 c $major 0
mknod /dev/${device}1 c $major 1
mknod /dev/${device}2 c $major 2
mknod /dev/${device}3 c $major 3

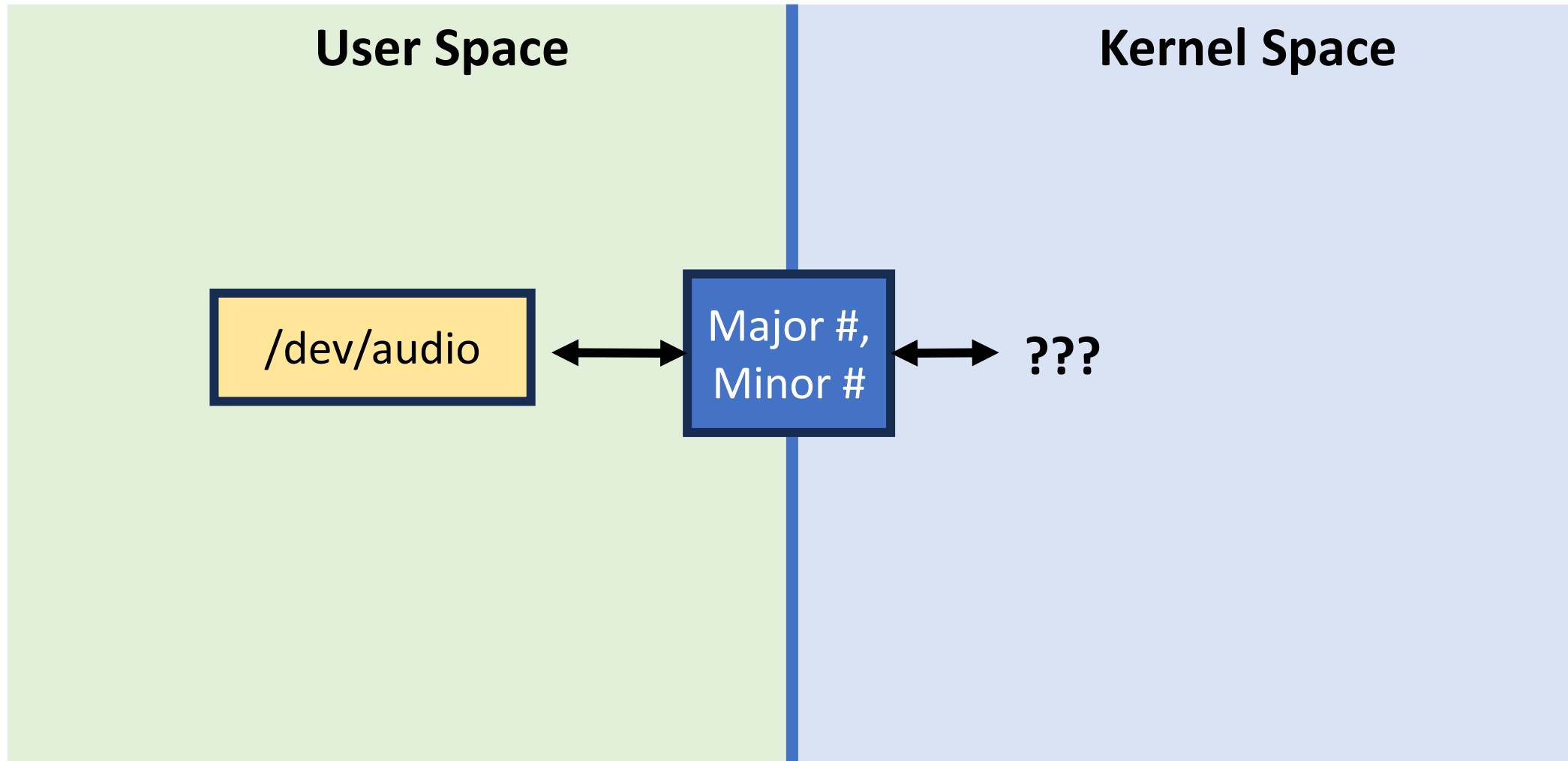
# give appropriate group/permissions, and change the group.
# Not all distributions have staff, some have "wheel" instead.
group="staff"
grep -q '^staff:' /etc/group || group="wheel"

chgrp $group /dev/${device}[0-3]
chmod $mode /dev/${device}[0-3]
```

The script can be adapted for another driver by redefining the variables and adjusting the *mknod* lines. The script just shown creates four devices because four is the default in the *scull* sources.

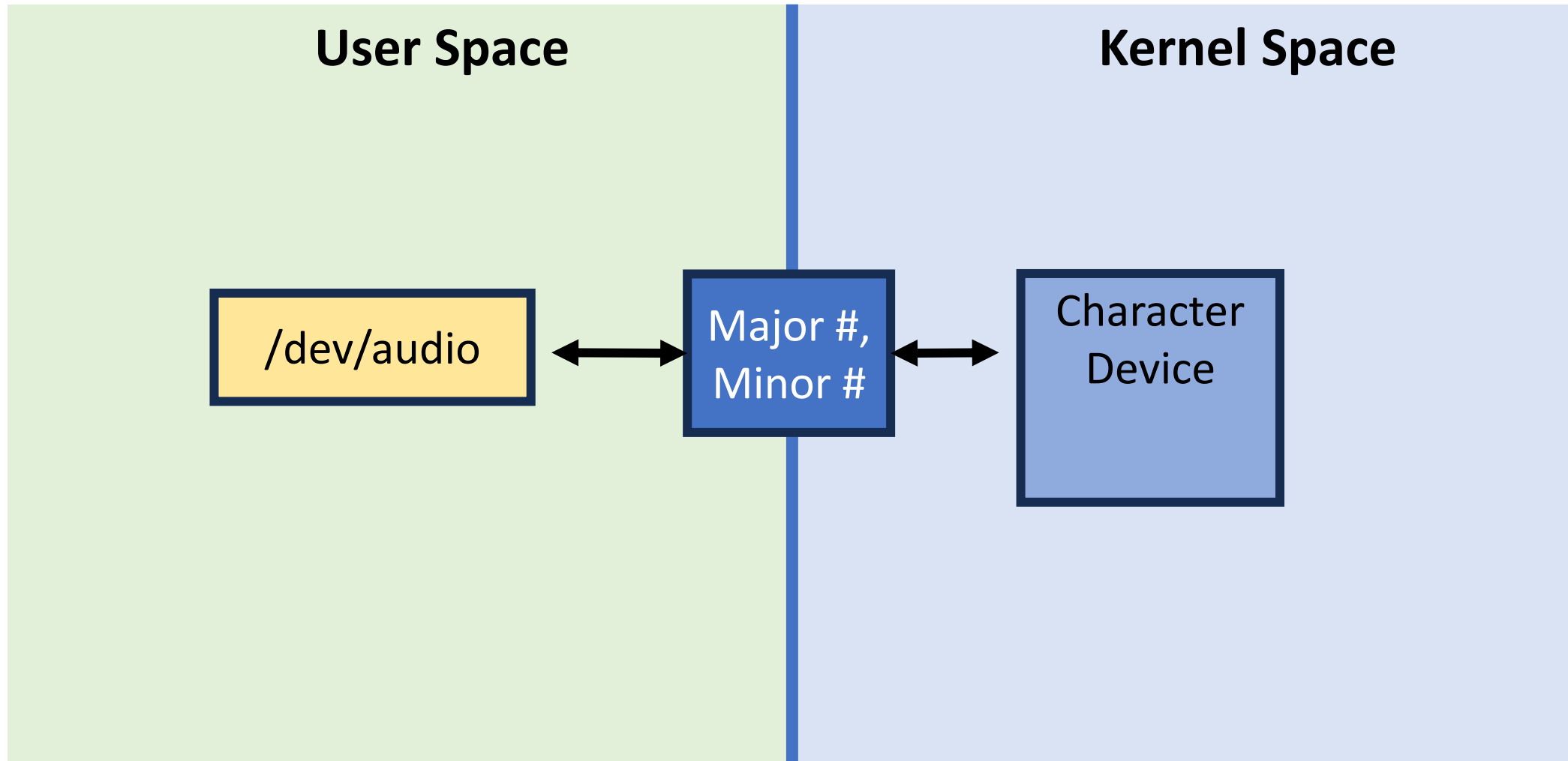
In the textbook, the device file is created using a “*scull\_load*” shell script called from user space.

- Uses **mknod**, which creates device files from user space
- In Lab 5, we will trigger the device file creation from within your driver
  - This is not shown in Chapter 3, but explained briefly in Ch 14 (with outdated function names), pp. 403-404



Next we need to tell Linux about our device...

We will create a new “character device” in the kernel



# Char Device Registration

There are two ways of allocating and initializing one of these structures. If you wish to obtain a standalone cdev structure at runtime, you may do so with code such as:

```
struct cdev *my_cdev = cdev_alloc();  
my_cdev->ops = &my_fops;
```

Chances are, however, that you will want to embed the cdev structure within a device-specific structure of your own; that is what *scull* does. In that case, you should initialize the structure that you have already allocated with:

```
void cdev_init(struct cdev *cdev, struct file_operations *fops);
```

<https://elixir.bootlin.com/linux/v6.6.10/source/include/linux/cdev.h#L14>

cdev\_init fills in everything except owner:

```
struct cdev;  
cdev_init(&cdev, ...);  
cdev.owner = THIS_MODULE;
```

# Adding cdev to Linux

Once the cdev structure is set up, the final step is to tell the kernel about it with a call to:

```
int cdev_add(struct cdev *dev, dev_t num, unsigned int count);
```

Here, dev is the cdev structure, num is the first device number to which this device responds, and count is the number of device numbers that should be associated with the device.

To remove a char device from the system, call:

```
void cdev_del(struct cdev *dev);
```

Clearly, you should not access the cdev structure after passing it to *cdev\_del*.

# A struct for all device data

Internally, *scull* represents each device with a structure of type `struct scull_dev`. This structure is defined as:

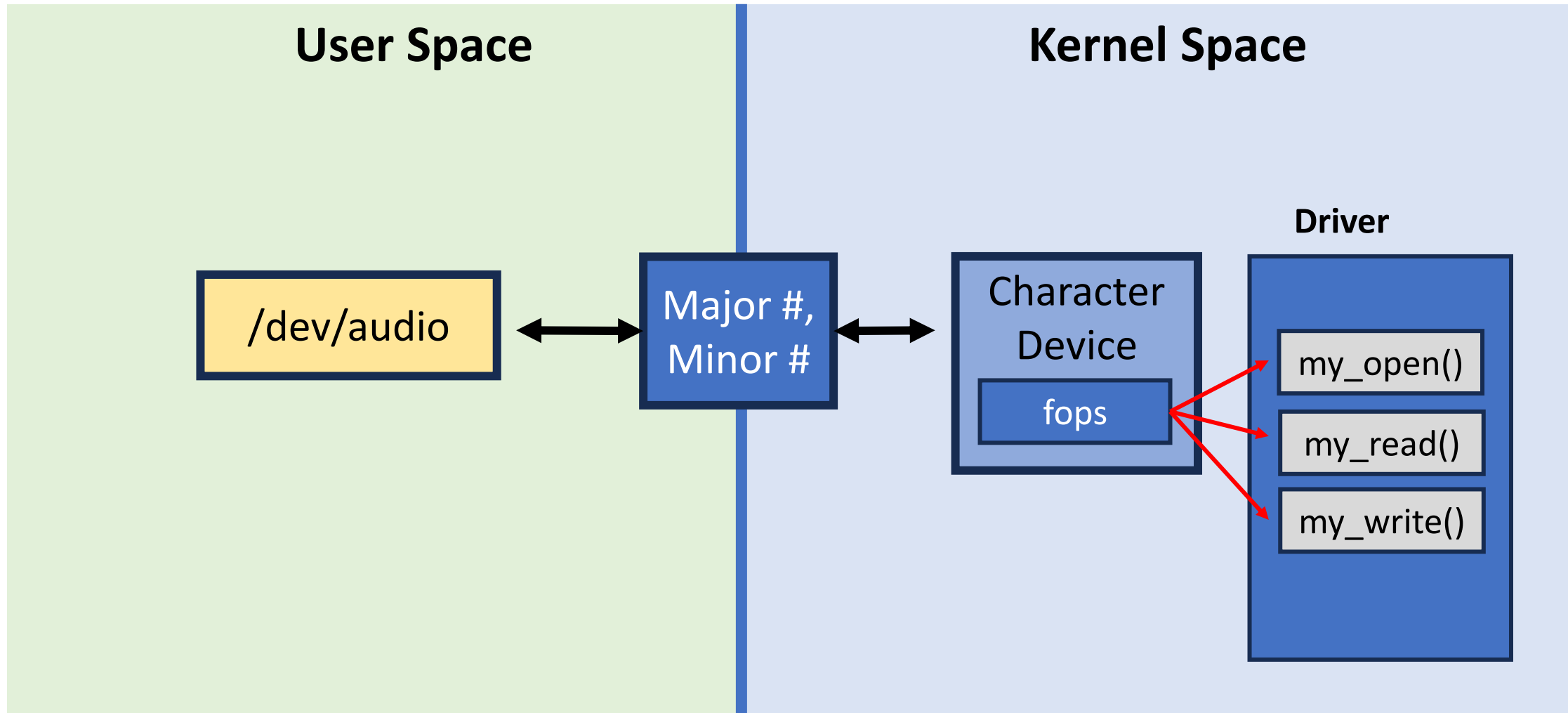
```
struct scull_dev {
    struct scull_qset *data; /* Pointer to first quantum set */
    int quantum;            /* the current quantum size */
    int qset;               /* the current array size */
    unsigned long size;     /* amount of data stored here */
    unsigned int access_key; /* used by sculluid and scullpriv */
    struct semaphore sem;   /* mutual exclusion semaphore */
    struct cdev cdev;       /* Char device structure */
};
```

For Lab 5, we will do something similar, and have our own “struct” for each device in the driver.

- (Our driver will only have 1 device) 😊

- The fops\* we provide to the character device contains a struct of pointers to different functions in our driver for any file operations we want to support.
- We can leave function pointers NULL for unsupported operations. “The exact behavior of the kernel when a NULL pointer is specified is different for each function”

<https://elixir.bootlin.com/linux/v6.6.10/source/include/linux/fs.h#L1911>



# Initializing struct file\_operations

Defined in  
linux/fs.h

```
struct file_operations {
    struct module *owner;
    ...
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
    ...
    int (*open) (struct inode *, struct file *);
    ...
};
```

In your driver, you declare an instance of this struct, populate it, and store a pointer to it in your character device.

```
struct file_operations scull_fops = {
    .owner = THIS_MODULE,
    .llseek = scull_llseek,
    .read = scull_read,
    .write = scull_write,
    .ioctl = scull_ioctl,
    .open = scull_open,
    .release = scull_release,
};
```

```
int scull_open(struct inode *inode,
               struct file *filp) {
    ...
}
```

Your function (e.g. `scull_open`), must be of the correct type.  
Remember: a function type is defined by its return value and argument types.

```
cdev_init(..., &scull_fops);
```

# struct file\_operations

struct module \*owner

The first `file_operations` field is not an operation at all; it is a pointer to the module that “owns” the structure. This field is used to prevent the module from being unloaded while its operations are in use. Almost all the time, it is simply initialized to `THIS_MODULE`, a macro defined in `<linux/module.h>`.

loff\_t (\*llseek) (struct file \*, loff\_t, int);

The `llseek` method is used to change the current read/write position in a file, and the new position is returned as a (positive) return value. The `loff_t` parameter is a “long offset” and is at least 64 bits wide even on 32-bit platforms. Errors are signaled by a negative return value. If this function pointer is `NULL`, seek calls will modify the position counter in the file structure (described in the section “The file Structure”) in potentially unpredictable ways.

ssize\_t (\*read) (struct file \*, char \_\_user \*, size\_t, loff\_t \*);

Used to retrieve data from the device. A null pointer in this position causes the `read` system call to fail with `-EINVAL` (“Invalid argument”). A nonnegative return value represents the number of bytes successfully read (the return value is a “signed size” type, usually the native integer type for the target platform).

ssize\_t (\*aio\_read)(struct kiocb \*, char \_\_user \*, size\_t, loff\_t);

Initiates an asynchronous read—a read operation that might not complete before the function returns. If this method is `NULL`, all operations will be processed (synchronously) by `read` instead.

The textbook has descriptions of the various fops methods, although some struct member names and types have changed slightly since the textbook kernel version.

# struct file\_operations

ssize\_t (\*write) (struct file \*, const char \_\_user \*, size\_t, loff\_t \*);

Sends data to the device. If NULL, -EINVAL is returned to the program calling the *write* system call. The return value, if nonnegative, represents the number of bytes successfully written.

int (\*ioctl) (struct inode \*, struct file \*, unsigned int, unsigned long);

The *ioctl* system call offers a way to issue device-specific commands (such as formatting a track of a floppy disk, which is neither reading nor writing). Additionally, a few *ioctl* commands are recognized by the kernel without referring to the fops table. If the device doesn't provide an *ioctl* method, the system call returns an error for any request that isn't predefined (-ENOTTY, "No such ioctl for device").

int (\*mmap) (struct file \*, struct vm\_area\_struct \*);

*mmap* is used to request a mapping of device memory to a process's address space. If this method is NULL, the *mmap* system call returns -ENODEV.

int (\*open) (struct inode \*, struct file \*);

Though this is always the first operation performed on the device file, the driver is not required to declare a corresponding method. If this entry is NULL, opening the device always succeeds, but your driver isn't notified.

int (\*release) (struct inode \*, struct file \*);

This operation is invoked when the file structure is being released. Like *open*, *release* can be NULL.\*

# The open function

```
int (*open)(struct inode *inode, struct file *filp);
```

struct inode:

- the kernel's record of the file in /dev. One per device file.
- Holds the major/minor and a pointer to the cdev you registered.

struct file:

- this particular open() of the file. One per file descriptor.
- Passed to every later read, write and ioctl.

Members:

- f\_mode: opened for read, write or both.
- f\_pos: current file position.
- private\_data: NULL on open, then yours. Set it in open(), read it back in read/write/ioctl.

# release

```
int scull_release(struct inode *inode, struct file *filp)
{
    return 0;
}
```

Release happens when you close a file.

The *read* and *write* methods both perform a similar task, that is, copying data from and to application code. Therefore, their prototypes are pretty similar, and it's worth introducing them at the same time:

```
ssize_t read(struct file *filp, char __user *buff,  
             size_t count, loff_t *offp);  
ssize_t write(struct file *filp, const char __user *buff,  
              size_t count, loff_t *offp);
```

For both methods, *filp* is the file pointer and *count* is the size of the requested data transfer. The *buff* argument points to the user buffer holding the data to be written or the empty buffer where the newly read data should be placed. Finally, *offp* is a pointer to a “long offset type” object that indicates the file position the user is accessing. The return value is a “signed size type”; its use is discussed later.

# read() and write() return values

```
ssize_t read(struct file *filp, char __user *buff, size_t count, loff_t *offp)
```

This should return the number of bytes you put in buff. It becomes the return value of the user's read() call.

- `== count`: everything they asked for
- `0 < n < count`: part of it. The program (or C library) usually calls read() again for the rest.
- `0`: end of file. Nothing now, nothing coming.
- `< 0`: error. User sees -1, with `errno` set to the value (-EFAULT, -EINVAL, ...).
- `*offp`: Depending on how your device works, you may need to increment this accordingly

```
ssize_t write(struct file *filp, const char __user *buff, size_t count, loff_t *offp)
```

Return value is the number of bytes you consumed from buff.

- Similar to above

Whatever the amount of data the methods transfer, they should generally update the file position at `*offp` to represent the current file position after successful completion of the system call. The kernel then propagates the file position change back into the file structure when appropriate.

# Why can't the driver just dereference buff?

Let us repeat that the *buff* argument to the *read* and *write* methods is a user-space pointer. Therefore, it cannot be directly dereferenced by kernel code. There are a few reasons for this restriction:

- Depending on which architecture your driver is running on, and how the kernel was configured, the user-space pointer may not be valid while running in kernel mode at all. There may be no mapping for that address, or it could point to some other, random data.
- Even if the pointer does mean the same thing in kernel space, user-space memory is paged, and the memory in question might not be resident in RAM when the system call is made. Attempting to reference the user-space memory directly could generate a page fault, which is something that kernel code is not allowed to do. The result would be an “oops,” which would result in the death of the process that made the system call.
- The pointer in question has been supplied by a user program, which could be buggy or malicious. If your driver ever blindly dereferences a user-supplied pointer, it provides an open doorway allowing a user-space program to access or overwrite memory anywhere in the system. If you do not wish to be responsible for compromising the security of your users' systems, you cannot ever dereference a user-space pointer directly.

# copy\_to\_user / copy\_from\_user

```
unsigned long copy_to_user(void __user *to,  
                           const void *from,  
                           unsigned long count);  
unsigned long copy_from_user(void *to,  
                             const void __user *from,  
                             unsigned long count);
```

“The role of the two functions is not limited to copying data to and from user-space: they also check whether the user space pointer is valid. If the pointer is invalid, no copy is performed; if an invalid address is encountered during the copy, on the other hand, only part of the data is copied. In both cases, the **return value is the amount of memory still to be copied**. The scull code looks for this error return, and returns-EFAULT to the user if it's not 0.”

# scull\_read example

```
ssize_t scull_read(struct file *filp, char __user *buf, size_t count,
                  loff_t *f_pos)
{
    struct scull_dev *dev = filp->private_data;
    struct scull_qset *dptr;    /* the first listitem */
    int quantum = dev->quantum, qset = dev->qset;
    int itemsize = quantum * qset; /* how many bytes in the listitem */
    int item, s_pos, q_pos, rest;
    ssize_t retval = 0;

    if (down_interruptible(&dev->sem))
        return -ERESTARTSYS;
    if (*f_pos >= dev->size)
        goto out;
    if (*f_pos + count > dev->size)
        count = dev->size - *f_pos;

    /* find listitem, qset index, and offset in the quantum */
    item = (long)*f_pos / itemsize;
    rest = (long)*f_pos % itemsize;
    s_pos = rest / quantum; q_pos = rest % quantum;

    /* follow the list up to the right position (defined elsewhere) */
    dptr = scull_follow(dev, item);

    if (dptr == NULL || !dptr->data || !dptr->data[s_pos])
        goto out; /* don't fill holes */

    /* read only up to the end of this quantum */
    if (count > quantum - q_pos)
        count = quantum - q_pos;

    if (copy_to_user(buf, dptr->data[s_pos] + q_pos, count)) {
        retval = -EFAULT;
        goto out;
    }
    *f_pos += count;
    retval = count;

out:
    up(&dev->sem);
    return retval;
}
```